

COMP0089: Reinforcement Learning

Xu Chen

April 7, 2026

Contents

1	Fundamentals of Reinforcement Learning	4
1.1	Returns and Discounting	4
1.2	Markov Decision Processes	4
1.3	Policies	4
2	Value Functions and the Bellman Equations	5
2.1	Value Functions	5
2.2	Bellman Equations	5
3	Theoretical Foundations: Operators and Fixed Points	6
3.1	Contraction Mappings and Fixed Point Theory	6
3.2	Bellman Operators	6
4	Planning: Dynamic Programming Algorithms	7
4.1	Policy Evaluation	7
4.2	Policy Improvement and Iteration	7
4.3	Value Iteration	8
5	Multi-Armed Bandits and Exploration	8
5.1	The Bandit Problem	8
5.2	Exploration-Exploitation Strategies	9
5.3	Optimism Under Uncertainty	10
6	Monte Carlo and Temporal Difference Learning	10
6.1	Monte Carlo Methods	10
6.2	Temporal Difference Learning	11
6.3	n-Step Methods and Eligibility Traces	11
7	Control Algorithms: Model-Free RL	12
7.1	On-Policy Control: SARSA	12
7.2	Off-Policy Control: Q-Learning	12
7.3	Expected SARSA	13
7.4	GLIE and Convergence	13

8	Function Approximation and Deep RL	14
8.1	Linear Value Function Approximation	14
8.2	Convergence of Approximation Methods	14
8.3	The Deadly Triad	14
8.4	Modern Stabilization Techniques	15
9	Model-Based Reinforcement Learning	16
9.1	Learning and Using Models	16
9.2	Dyna and Integrated Learning	16
9.3	Monte Carlo Tree Search	17
10	Policy Gradient Methods	17
10.1	Policy Parameterization and Gradient Theorem	17
10.2	The REINFORCE Algorithm	18
10.3	Actor-Critic Methods	18
10.4	Trust Region and Deterministic Policy Gradients	18
11	Off-Policy Learning and Importance Sampling	19
11.1	Importance Sampling Corrections	19
11.2	Variance Reduction Techniques	19
11.3	Tree Backup and Retrace	20
12	Deep Reinforcement Learning	20
12.1	Deep Q-Networks	20
12.2	Advanced DQN Methods	20
12.3	Gaussian Policies for Continuous Control	21
13	Final Remarks	21

Introduction

These lecture notes provide a comprehensive treatment of reinforcement learning, covering the fundamental concepts of Markov decision processes, value functions, temporal difference methods, and policy gradient approaches. The material progresses from basic theoretical foundations through practical learning algorithms to advanced deep reinforcement learning techniques.

1 Fundamentals of Reinforcement Learning

1.1 Returns and Discounting

Definition 1.1 (Return). The return G_t is the cumulative future reward starting from time step t :

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

In undiscounted settings, this is simply the sum of all future rewards until the episode ends.

Definition 1.2 (Discounted Return with Discount Factor). The discounted return incorporates a discount factor $\gamma \in [0, 1)$ that weights immediate rewards more heavily than future rewards:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

The discount factor determines the time horizon for decision making. When γ is close to 1, the agent values long-term rewards; when γ is close to 0, the agent prioritizes immediate rewards.

1.2 Markov Decision Processes

Definition 1.3 (Markov Decision Process (MDP)). A Markov decision process is a tuple (S, A, p, r, γ) where:

- S is the set of all possible states
- A is the set of all possible actions
- $p(s'|s, a)$ is the transition probability: $P(S_{t+1} = s' | S_t = s, A_t = a)$
- $r(s, a)$ is the expected reward function: $\mathbb{E}[R_{t+1} | S_t = s, A_t = a]$
- $\gamma \in [0, 1)$ is the discount factor

The key defining property of an MDP is the Markov property: future states depend only on the current state and action, not on the history:

$$P(S_{t+1} = s' | S_t = s, A_t = a) = P(S_{t+1} = s' | S_t = s, A_t = a, \text{history})$$

This memoryless property allows for efficient dynamic programming solutions.

1.3 Policies

Definition 1.4 (Policy). A policy is a mapping $\pi : S \times A \rightarrow [0, 1]$ that assigns probabilities $\pi(a|s)$ to actions in each state. A policy describes how the agent behaves:

- **Stochastic policy:** $\pi(a|s)$ can be any probability distribution over actions
- **Deterministic policy:** $\pi(a|s) \in \{0, 1\}$, selecting a single action per state

Definition 1.5 (Goals of Reinforcement Learning). The goal of an RL agent is to find a behavior policy that maximizes the expected return:

$$\pi^* = \arg \max_{\pi} \mathbb{E}[G_0 | \pi]$$

An optimal policy achieves the highest expected cumulative reward from any starting state.

2 Value Functions and the Bellman Equations

2.1 Value Functions

Definition 2.1 (State Value Function). The state value function $V_{\pi}(s)$ is the expected cumulative discounted reward obtained by starting in state s and following policy π :

$$V_{\pi}(s) = \mathbb{E}[G_t | S_t = s, \pi] = \mathbb{E}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) | S_t = s]$$

It represents how good it is to be in a given state under a particular policy.

Definition 2.2 (Action Value Function (Q-Function)). The action value function $Q_{\pi}(s, a)$ is the expected return from taking action a in state s and then following policy π :

$$Q_{\pi}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a, \pi] = \mathbb{E}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) | S_t = s, A_t = a]$$

The relationship between V and Q under policy π is:

$$V_{\pi}(s) = \sum_a \pi(a|s) Q_{\pi}(s, a)$$

2.2 Bellman Equations

Theorem 2.1 (Bellman Expectation Equations). For any policy π , the value functions satisfy the Bellman equations:

State-Value Bellman Equation:

$$V_{\pi}(s) = \sum_a \pi(a|s) \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V_{\pi}(s') \right]$$

Action-Value Bellman Equation:

$$Q_{\pi}(s, a) = r(s, a) + \gamma \sum_{s'} p(s'|s, a) \sum_{a'} \pi(a'|s') Q_{\pi}(s', a')$$

These recursive equations decompose the value of a state into the immediate reward plus the discounted value of successor states.

Definition 2.3 (Optimal Policy). An optimal policy π^* is any policy that achieves the maximum expected return from every state:

$$\pi^*(a|s) = \begin{cases} 1 & \text{if } a = \arg \max_a Q^*(s, a) \\ 0 & \text{otherwise} \end{cases}$$

Key fact: For any finite MDP, there exists at least one deterministic optimal policy.

Theorem 2.2 (Bellman Optimality Equations). The optimal value functions satisfy the Bellman optimality equations:

Optimal State-Value:

$$V^*(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V^*(s') \right]$$

Optimal Action-Value:

$$Q^*(s, a) = r(s, a) + \gamma \sum_{s'} p(s'|s, a) \max_{a'} Q^*(s', a')$$

Once we have the optimal value functions, the optimal policy is:

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

3 Theoretical Foundations: Operators and Fixed Points

3.1 Contraction Mappings and Fixed Point Theory

Definition 3.1 (Contraction Mapping). Let X be a vector space equipped with a norm $\|\cdot\|$. A mapping $T : X \rightarrow X$ is a γ -contraction if for all $x_1, x_2 \in X$:

$$\|Tx_1 - Tx_2\| \leq \gamma \|x_1 - x_2\|$$

where $\gamma \in [0, 1]$ is the contraction coefficient. A contraction mapping brings points closer together under repeated application.

Theorem 3.1 (Banach Fixed Point Theorem). Let X be a complete normed vector space and $T : X \rightarrow X$ a γ -contraction mapping. Then:

1. T has a unique fixed point $x^* \in X$ such that $Tx^* = x^*$
2. For any starting point $x_0 \in X$, the sequence $x_{n+1} = Tx_n$ converges to x^* geometrically:

$$\|x_n - x^*\| \leq \gamma^n \|x_0 - x^*\|$$

This theorem guarantees convergence of iterative algorithms based on contraction mappings.

3.2 Bellman Operators

Definition 3.2 (Bellman Optimality Operator). For a bounded function space, the Bellman optimality operator T^* is defined pointwise as:

$$(T^*f)(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) f(s') \right]$$

Key properties:

- T^* is a γ -contraction with respect to the L^∞ norm

- The unique fixed point of T^* is V^*
- Value iteration $V_{k+1} = T^*V_k$ converges exponentially to V^*

Definition 3.3 (Bellman Expectation Operator). For a given policy π , the Bellman expectation operator T^π is defined as:

$$(T^\pi f)(s) = \sum_a \pi(a|s) \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) f(s') \right]$$

Like T^* , the operator T^π is a γ -contraction, and policy evaluation iterates converge to V^π .

4 Planning: Dynamic Programming Algorithms

4.1 Policy Evaluation

Definition 4.1 (Policy Evaluation (Prediction)). Given a policy π , policy evaluation estimates the value function V_π through iterative application of the Bellman expectation equation:

Algorithm:

- Initialize $V(s) = 0$ for all $s \in S$
- Repeat:

$$V_{k+1}(s) \leftarrow \sum_a \pi(a|s) \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V_k(s') \right]$$

- Until convergence: $\max_s |V_{k+1}(s) - V_k(s)| < \theta$

By the Banach fixed point theorem, this converges to V^π .

4.2 Policy Improvement and Iteration

Theorem 4.1 (Policy Improvement Theorem). Given value estimates V_k for policy π_k , if a new greedy policy is defined as:

$$\pi_{k+1}(s) = \arg \max_a \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V_k(s') \right]$$

then the new policy is at least as good as the old policy: $V^{\pi_{k+1}}(s) \geq V^{\pi_k}(s)$ for all states s .

Definition 4.2 (Policy Iteration). Policy iteration alternates between policy evaluation and policy improvement:

Algorithm:

1. Initialize policy π_0 arbitrarily
2. Repeat until convergence:

- *Evaluation*: Compute V^{π_k} using policy evaluation
- *Improvement*: $\pi_{k+1}(s) = \arg \max_a Q^{\pi_k}(s, a)$

Policy iteration is guaranteed to converge to the optimal policy π^* and optimal value function V^* .

4.3 Value Iteration

Definition 4.3 (Value Iteration). Value iteration combines policy evaluation and improvement into a single update step:

Algorithm:

- Initialize $V(s) = 0$ for all s
- Repeat until convergence:

$$V_{k+1}(s) \leftarrow \max_a \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V_k(s') \right]$$

- Extract greedy policy: $\pi(s) = \arg \max_a Q^*(s, a)$

This is equivalent to applying the Bellman optimality operator: $V_{k+1} = T^*V_k$. Convergence is guaranteed by contraction mapping theory.

5 Multi-Armed Bandits and Exploration

5.1 The Bandit Problem

Definition 5.1 (Multi-Armed Bandit Problem). In the k -armed bandit problem, an agent selects from k actions at each step, each with an unknown reward distribution. Unlike MDPs, there is only one state, and actions don't affect future states.

Key quantities:

- $Q^*(a) = \mathbb{E}[R_t | A_t = a]$ is the true value of action a
- $Q_t(a)$ is the estimated value after t steps
- $\Delta_a = Q^*(a_*) - Q^*(a)$ is the regret of action a relative to the optimal action a_*

The goal is to maximize cumulative reward, or equivalently, minimize total regret:

$$R_T = \sum_{t=1}^T (Q^*(a_*) - Q^*(A_t)) = \sum_{a \neq a_*} \Delta_a N_T(a)$$

5.2 Exploration-Exploitation Strategies

Definition 5.2 (Greedy and ε -Greedy). **Greedy action selection** always picks the action with highest estimated value:

$$A_t = \arg \max_a Q_t(a)$$

This is optimal if estimates are accurate, but poor during learning when estimates are uncertain.

ε -Greedy strategy balances exploration and exploitation:

$$A_t = \begin{cases} \arg \max_a Q_t(a) & \text{with probability } 1 - \varepsilon \\ \text{random action} & \text{with probability } \varepsilon \end{cases}$$

The ε -greedy strategy is simple, but exploration rate doesn't adapt: it explores equally regardless of how much uncertainty remains.

Definition 5.3 (Hoeffding's Inequality). Let $X_1, \dots, X_n$ be i.i.d. random variables in $[0, 1]$ with mean $\mu = \mathbb{E}[X]$. Let $\bar{X}_n = \frac{1}{n} \sum_i X_i$ be the sample mean. Then:

$$P(\bar{X}_n - \mu > u) \leq e^{-2nu^2}$$

This concentration bound shows that sample averages are unlikely to deviate far from their true mean with high probability.

Definition 5.4 (Upper Confidence Bound (UCB) Algorithm). The UCB algorithm selects actions based on both their estimated value and uncertainty:

$$A_t = \arg \max_{a \in A} [Q_t(a) + U_t(a)]$$

where $U_t(a) = \sqrt{\frac{2 \log t}{N_t(a)}}$ is the confidence radius. Actions with:

- High estimated value are preferred (exploitation)
- High uncertainty are tried to reduce uncertainty (exploration)

UCB achieves logarithmic regret: $R_T = O(\log T)$, which is asymptotically optimal.

Definition 5.5 (Thompson Sampling). Thompson sampling maintains a posterior distribution over the value of each action and samples from these posteriors:

Algorithm:

- Maintain belief $p_t(Q(a))$ about each action value
- Sample $\tilde{Q}_t(a) \sim p_t(Q(a))$ for each action
- Select $A_t = \arg \max_a \tilde{Q}_t(a)$
- After observing reward, update belief using Bayes' rule

Thompson sampling naturally balances exploration and exploitation by sampling from the posterior. It achieves logarithmic regret and is often more efficient than UCB in practice.

5.3 Optimism Under Uncertainty

Remark 5.1 (Exploration-Exploitation Trade-off). When choosing actions, we balance:

- **Exploitation:** Choose actions with high estimated value μ_a
- **Exploration:** Choose actions with high uncertainty σ_a to reduce uncertainty

Both UCB and Thompson sampling implement the principle of *optimism under uncertainty*: favor actions that could plausibly be optimal given current beliefs.

Theorem 5.1 (Lai and Robbins Lower Bound). No algorithm can achieve regret asymptotically better than:

$$\lim_{T \rightarrow \infty} \frac{\mathbb{E}[R_T]}{\log T} \geq \sum_{a \neq a_*} \frac{\Delta_a}{\text{KL}(Q^*(a) || Q^*(a_*))}$$

This shows that algorithms like UCB and Thompson sampling that achieve $O(\log T)$ regret are fundamentally optimal.

6 Monte Carlo and Temporal Difference Learning

6.1 Monte Carlo Methods

Definition 6.1 (Monte Carlo Policy Evaluation). Monte Carlo estimation computes returns from complete episodes and averages them to estimate value functions:

Algorithm:

- Generate episodes under policy π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T$
- For each state visited, compute the return: $G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T$
- Update average: $V(S_t) \leftarrow \frac{1}{N} \sum_i G_t^{(i)}$

Advantages:

- Returns are unbiased estimates of $V_\pi(S_t)$
- Converges to true value even without knowing transition probabilities
- No bootstrapping - doesn't use value estimates

Disadvantages:

- Only works for episodic tasks
- High variance: must wait for full episode to complete

6.2 Temporal Difference Learning

Definition 6.2 (Temporal Difference (TD) Learning). TD methods combine ideas from Monte Carlo and dynamic programming. Instead of waiting for complete returns, they use bootstrapping to update value estimates:

TD(0) Update Rule:

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

where $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ is the TD error.

Advantages:

- Low variance: uses value estimate bootstrapping
- Incremental: learns from each transition
- Works for continuing tasks

Disadvantages:

- Biased estimate of $V_\pi(S_t)$ unless $V(S_{t+1})$ is accurate
- Depends on accurate value estimates

Remark 6.1 (MC vs TD Trade-off). • **MC**: Unbiased but high variance - converges more slowly

- **TD**: Biased but low variance - often learns faster with finite data
- **Batch learning difference**: With finite data, TD converges to maximum likelihood estimate of transition model, while MC minimizes error on observed returns

6.3 n-Step Methods and Eligibility Traces

Definition 6.3 (n-Step Returns). The n-step return generalizes between TD(0) and Monte Carlo:

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

Values of n :

- $n = 1$: One-step TD update
- $n = \infty$: Pure Monte Carlo return (if episode ends)
- $1 < n < \infty$: Intermediate estimates

The update becomes:

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t^{(n)} - V(S_t)]$$

Definition 6.4 (λ -Return and $\text{TD}(\lambda)$). The λ -return is a weighted mixture of n -step returns, emphasizing shorter steps as $\lambda \rightarrow 0$ and longer steps as $\lambda \rightarrow 1$:

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

This can be rewritten as:

$$G_t^\lambda = R_{t+1} + \gamma((1 - \lambda)V(S_{t+1}) + \lambda G_{t+1}^\lambda)$$

$\text{TD}(\lambda)$ uses the λ -return to update: $V(S_t) \leftarrow V(S_t) + \alpha[G_t^\lambda - V(S_t)]$

Definition 6.5 (Eligibility Traces). Eligibility traces provide an efficient way to implement $\text{TD}(\lambda)$ by replacing the λ -return with trace-based updates. Define the eligibility trace as:

$$e_t(s) = \begin{cases} \gamma \lambda e_{t-1}(s) & \text{if } S_t \neq s \\ \gamma \lambda e_{t-1}(s) + 1 & \text{if } S_t = s \end{cases}$$

Then the update becomes:

$$V(s) \leftarrow V(s) + \alpha \delta_t e_t(s) \quad \text{for all } s$$

where $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ is the TD error. Traces decay over time, implementing the λ -return efficiently.

7 Control Algorithms: Model-Free RL

7.1 On-Policy Control: SARSA

Definition 7.1 (SARSA (State-Action-Reward-State-Action)). SARSA is an on-policy TD method that learns the value of the policy being followed:

Algorithm:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

where A_{t+1} is the action actually taken in the next step under the current policy. Properties:

- On-policy: Learns the value of the exploration policy being followed
- Uses the next action A_{t+1} that was actually taken
- Converges to optimal Q-values for the exploration policy, not the optimal policy

7.2 Off-Policy Control: Q-Learning

Definition 7.2 (Q-Learning). Q-learning is an off-policy TD method that learns the optimal action-value function regardless of which policy is being followed:

Algorithm:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)]$$

Key differences from SARSA:

- Off-policy: Learns optimal policy π^* while following exploration policy μ
- Uses the maximum next Q-value instead of the action actually taken
- Learns the optimal value regardless of exploratory actions

Q-learning is widely used because it separates the exploration policy (for gathering data) from the target policy (being optimized).

Definition 7.3 (Double Q-Learning). Standard Q-learning can overestimate action values because it uses $\max_a Q(S_{t+1}, a)$, which may be an optimistic estimate when values are learned. Double Q-learning reduces this bias by decoupling action selection from evaluation:

Algorithm:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, \arg \max_{a'} Q(S_{t+1}, a')) - Q(S_t, A_t)]$$

Or, maintain two Q-networks Q and Q' :

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q'(S_{t+1}, \arg \max_{a'} Q(S_{t+1}, a')) - Q(S_t, A_t)]$$

This separates the actions selected (via Q) from those evaluated (via Q'), reducing overestimation bias.

7.3 Expected SARSA

Definition 7.4 (Expected SARSA). Expected SARSA combines on-policy structure with off-policy bootstrap by taking the expectation over actions:

Algorithm:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_{a'} \pi(a'|S_{t+1}) Q(S_{t+1}, a') - Q(S_t, A_t) \right]$$

where π is the target policy (often greedy, sometimes ε -greedy).

This uses the expected value under the target policy rather than bootstrapping from a single next action, combining benefits of SARSA and Q-learning.

7.4 GLIE and Convergence

Definition 7.5 (GLIE (Greedy in the Limit with Infinite Exploration)). A learning process is GLIE if:

1. All state-action pairs are visited infinitely often: $\lim_{t \rightarrow \infty} N_t(s, a) = \infty$ for all s, a
2. The policy converges to greedy: $\lim_{t \rightarrow \infty} \pi_t(a|s) = \mathbb{1}_{a=\arg \max_a Q_t(s,a)}$

Example: ε -greedy with $\varepsilon_t = \frac{1}{t}$ satisfies GLIE.

Theorem 7.1. If Q-learning is performed with:

- GLIE exploration (e.g., $\varepsilon_t \rightarrow 0$ decay)
- Decaying step sizes: $\sum_t \alpha_t = \infty$ and $\sum_t \alpha_t^2 < \infty$

Then Q-learning converges to Q^* with probability 1.

8 Function Approximation and Deep RL

8.1 Linear Value Function Approximation

Definition 8.1 (Linear Function Approximation). Instead of maintaining a table of values, we use a linear combination of features:

$$\hat{V}_w(s) = \mathbf{w}^\top \mathbf{x}(s)$$

where $\mathbf{x}(s) \in \mathbb{R}^m$ is a feature vector and $\mathbf{w} \in \mathbb{R}^m$ are the weights.

Gradient-Based Updates: The gradient of the value function with respect to weights is:

$$\nabla_w \hat{V}_w(s) = \mathbf{x}(s)$$

For TD learning:

$$\Delta w = \alpha \delta_t \mathbf{x}(S_t)$$

where $\delta_t = R_{t+1} + \gamma \hat{V}_w(S_{t+1}) - \hat{V}_w(S_t)$ is the TD error.

Definition 8.2 (Coarse Coding). A simple feature representation using overlapping binary features:

$$\mathbf{x}(s) = [\mathbb{I}_{s \in R_1}, \mathbb{I}_{s \in R_2}, \dots, \mathbb{I}_{s \in R_m}]^\top$$

where each group R_i is a receptive field. If a state falls in a field, the corresponding feature is 1. This provides local generalization: similar states within the same field share weight updates.

8.2 Convergence of Approximation Methods

Definition 8.3 (MC and TD Convergence with Function Approximation). With linear function approximation and decreasing step sizes, MC and TD learning converge:

MC convergence target:

$$w_* = \mathbb{E}[(V_\pi(S) - \mathbf{w}^\top \mathbf{x}(S))^2]^{-1} \mathbb{E}[V_\pi(S) \mathbf{x}(S)]$$

Converges to minimize expected squared error on observed returns.

TD convergence target:

$$w_* = \mathbb{E}[\mathbf{x}(S)(\mathbf{x}(S) - \gamma \mathbf{x}(S'))^\top]^{-1} \mathbb{E}[\mathbf{x}(S) R_{t+1}]$$

This is the fixed point of the TD algorithm, not necessarily the best fit to true values but a fixed point of the iterative process.

8.3 The Deadly Triad

Definition 8.4 (The Deadly Triad). When three conditions occur together, RL algorithms can diverge:

1. **Bootstrapping:** Using value estimates to update other value estimates
2. **Off-Policy Learning:** Learning the optimal policy while following a different exploration policy

3. **Function Approximation:** Using a parameterized function approximator instead of lookup tables

Any two of these three can coexist safely, but all three together create instability. This is why:

- Tabular Q-learning + bootstrapping + off-policy is stable
- Tabular TD(λ) + bootstrapping + on-policy is stable
- DQN requires target networks and replay buffers to stabilize the combination

8.4 Modern Stabilization Techniques

Definition 8.5 (Experience Replay). Store transitions $(S_t, A_t, R_{t+1}, S_{t+1})$ in a replay buffer $\mathcal{D}$ and sample mini-batches uniformly at random for learning. This breaks temporal correlations and improves data efficiency.

Algorithm:

- Store every transition in buffer: $\mathcal{D} \leftarrow \mathcal{D} \cup \{(S_t, A_t, R_{t+1}, S_{t+1})\}$
- Sample batch $B \sim \mathcal{D}$ uniformly
- Update Q -network using batch: $L = \frac{1}{|B|} \sum_{i \in B} (R_i + \gamma \max_{a'} Q(S'_i, a') - Q(S_i, A_i))^2$

Definition 8.6 (Target Networks). Maintain a separate target network Q' (with parameters θ') to compute target values, while updating the main network Q (with parameters θ). The target network is updated periodically by copying weights from the main network: $\theta' \leftarrow \theta$ every C steps.

Update with target network:

$$y_i = R_{i+1} + \gamma \max_{a'} Q'(S'_i, a'; \theta')$$

where the target value y_i uses the old target network, reducing correlation between the target and the parameters being updated.

Definition 8.7 (Least Squares Temporal Difference (LSTD)). For batch learning with linear function approximation, LSTD solves for the exact TD fixed point:

$$w_{\text{LSTD}} = \left(\sum_{t=0}^T \mathbf{x}_t (\mathbf{x}_t - \gamma \mathbf{x}_{t+1})^\top \right)^{-1} \sum_{t=0}^T \mathbf{x}_t R_{t+1}$$

Define:

- $A_t = \sum_{t=0}^T \mathbf{x}_t (\mathbf{x}_t - \gamma \mathbf{x}_{t+1})^\top$
- $b_t = \sum_{t=0}^T \mathbf{x}_t R_{t+1}$

Then $w_{\text{LSTD}} = A_t^{-1} b_t$. These can be updated incrementally using matrix inversion lemmas.

9 Model-Based Reinforcement Learning

9.1 Learning and Using Models

Definition 9.1 (World Model). A model $\mathcal{M}$ approximates the environment dynamics:

$$\mathcal{M}(s, a) \approx P(S'|s, a), \quad \mathcal{R}(s, a) \approx \mathbb{E}[R|s, a]$$

The model can be:

- **Deterministic:** Predicts single next state and reward
- **Stochastic:** Predicts distribution over next states
- **Tabular:** Stores explicit tables of transitions and rewards
- **Parametric:** Uses function approximation (e.g., neural networks)

Definition 9.2 (Planning). Using a learned model to improve the value function and policy without interacting with the real environment. Key approaches:

- **Value iteration:** Apply $V_{k+1} = T^*V_k$ using the model
- **Policy iteration:** Alternate between policy evaluation and improvement using the model
- **Trajectory sampling:** Generate simulated trajectories and apply model-free algorithms

9.2 Dyna and Integrated Learning

Definition 9.3 (Dyna-Q). Dyna-Q integrates learning and planning by combining:
Real Experience:

1. Execute action a in environment, observe r, s'
2. Update Q-value: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$
3. Update model: $\mathcal{M}(s, a) \rightarrow s', r$

Planning (using model):

1. Sample random state-action pair (s, a) visited before
2. Simulate model: $s', r \sim \mathcal{M}(s, a)$
3. Update Q-value: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$
4. Repeat N times per environment interaction

Dyna uses the large amount of cheap simulated experience to amplify learning from real interactions.

9.3 Monte Carlo Tree Search

Definition 9.4 (Monte Carlo Tree Search (MCTS)). MCTS builds a partial search tree by repeatedly simulating episodes. Each iteration consists of four phases:

1. **Selection:** Traverse tree from root using a tree policy (e.g., UCB) until reaching a leaf
2. **Expansion:** Expand tree by adding a child node
3. **Rollout:** Simulate trajectory from expanded node using default policy until episode end
4. **Backup:** Propagate return back through selected path, updating visit counts and values:

$$Q(s, a) = \frac{\sum_{k=1}^N R^{(k)}(s, a) \mathbb{1}_{\{s, a \text{ visited in } k\}}}{N(s, a)}$$

MCTS are state-of-the-art for many games (Go, Chess) without learning a value function. The tree grows adaptively based on promising actions.

10 Policy Gradient Methods

10.1 Policy Parameterization and Gradient Theorem

Definition 10.1 (Policy Parameterization). A policy can be parameterized by a vector θ of parameters:

$$\pi_{\theta}(a|s) = \frac{e^{H_{\theta}(s,a)}}{\sum_b e^{H_{\theta}(s,b)}}$$

Softmax policy: Shows the probability of each action. For continuous control:

$$\pi_{\theta}(a|s) = \mathcal{N}(\mu_{\theta}(s), \sigma^2)$$

where $\mu_{\theta}(s)$ is the mean action learned by the neural network.

Theorem 10.1 (Policy Gradient Theorem (Episodic)). For episodic tasks with episode returns, the policy gradient is:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{S_t \sim \pi, A_t \sim \pi} [\nabla_{\theta} \log \pi_{\theta}(A_t|S_t) Q^{\pi_{\theta}}(S_t, A_t)]$$

The policy gradient is proportional to the log-probability of actions weighted by their Q-values. Actions with positive advantage get higher probability; those with negative advantage get lower probability.

Theorem 10.2 (Policy Gradient Theorem (Average Reward)). For continuing tasks with average reward, the gradient becomes:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{S_t \sim \mu_{\pi}, A_t \sim \pi} [\nabla_{\theta} \log \pi_{\theta}(A_t|S_t) Q^{\pi_{\theta}}(S_t, A_t)]$$

where $\mu_{\pi}(s)$ is the stationary state distribution under π .

10.2 The REINFORCE Algorithm

Definition 10.2 (REINFORCE). REINFORCE uses policy gradients to directly optimize the policy. The simplest form uses Monte Carlo returns:

Algorithm:

1. Generate episode under π_θ : $S_0, A_0, R_1, \dots, S_T$
2. For each step t , compute return: $G_t = \sum_{k=t}^{T-1} \gamma^{k-t} R_{k+1}$
3. Update policy:

$$\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(A_t|S_t) G_t$$

REINFORCE is simple but has high variance because each update uses only one return. Reductions include:

- Baseline subtraction: Use $G_t - b(S_t)$ instead of just G_t
- Advantage estimation: Use temporal difference advantages

10.3 Actor-Critic Methods

Definition 10.3 (Actor-Critic Architecture). Actor-critic methods maintain:

- **Actor:** The policy $\pi_\theta(a|s)$ that selects actions
- **Critic:** A value function $V_\phi(s)$ that estimates $V(s)$

The critic provides a baseline for policy gradient updates, reducing variance while maintaining unbiasedness (under TD(λ) with $\lambda = 1$).

Definition 10.4 (Advantage Actor-Critic (A2C)). Uses advantage function $A(s, a) = Q(s, a) - V(s)$ for policy updates:

- Critic TD error: $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ approximates advantage
- Actor update: $\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(A_t|S_t) \delta_t$
- Critic update: $\phi \leftarrow \phi + \beta \delta_t \nabla_\phi V_\phi(S_t)$

The critic learns faster (one-step TD), while the actor uses the critic's estimates to guide policy updates. This significantly reduces variance compared to REINFORCE.

10.4 Trust Region and Deterministic Policy Gradients

Definition 10.5 (Trust Regions). Policy updates should be conservative to avoid destabilizing the entire policy. One approach constrains the KL divergence between old and new policies:

$$\text{maximize } J(\theta) \quad \text{subject to} \quad \text{KL}(\pi_{\text{old}} || \pi_\theta) \leq \delta$$

This can be approximated with a penalty term in the objective, giving algorithms like PPO (Proximal Policy Optimization) which are robust and sample-efficient.

Definition 10.6 (Deterministic Policy Gradient). For continuous control, instead of sampling actions from a stochastic policy, use a deterministic policy $\mu_\theta(s)$ that outputs actions directly. The deterministic policy gradient is:

$$\nabla_\theta J = \mathbb{E}_S[\nabla_a Q(S, a)|_{a=\mu_\theta(S)} \cdot \nabla_\theta \mu_\theta(S)]$$

This decomposes into:

- $\nabla_a Q(S, a)$: How to improve actions
- $\nabla_\theta \mu_\theta(S)$: How to change the policy

Used in algorithms like Deep Deterministic Policy Gradient (DDPG). Works well when action space is mainly continuous.

11 Off-Policy Learning and Importance Sampling

11.1 Importance Sampling Corrections

Definition 11.1 (Off-Policy Learning). In off-policy learning, we learn the target policy π while following a different behavior policy μ . We need to correct for this distribution mismatch using importance sampling weights:

$$\rho_t = \frac{\pi(A_t|S_t)}{\mu(A_t|S_t)}$$

The importance sampling ratio tells us how much more likely the action was under the target policy vs the behavior policy.

Definition 11.2 (Per-Decision Importance Sampling). Instead of using full trajectory importance weights (which can explode), we can use per-decision importance sampling that limits corrections to individual steps:

Corrected Return:

$$G_t^c = \rho_t(R_{t+1} + \gamma(\rho_{t+1}G_{t+1}^c))$$

This avoids the exponential blowup of full importance sampling by only correcting over visited actions.

11.2 Variance Reduction Techniques

Definition 11.3 (Control Variates). Subtract a baseline function $b(S_t)$ from advantages to reduce variance without introducing bias:

$$A_t = (R_{t+1} + \gamma V(S_{t+1}) - V(S_t)) - b(S_t) + b(S_t) = \delta_t$$

The baseline should be highly correlated with returns to effectively reduce variance. Value function estimates are natural choices since they predict expected returns.

Definition 11.4 (Adaptive Bootstrapping). When importance sampling ratios ρ_t are large, importance sampling has high variance. Adaptive bootstrapping clips the correction:

$$\hat{A}_t = (1 - \lambda_t) [R_{t+1} + \gamma V(S_{t+1})] + \lambda_t \rho_t A_t$$

where $\lambda_t = \min(1, 1/\rho_t)$. This transitions from bootstrapping when ρ_t is large (high importance sampling variance) to importance sampling when ρ_t is small.

11.3 Tree Backup and Retrace

Definition 11.5 (Tree Backup). Tree backup is an off-policy algorithm that backs up values through expected targets:

$$G_t = R_{t+1} + \gamma \left(\sum_{a'} \pi(a'|S_{t+1}) Q(S_{t+1}, a') \cdot \pi(A_{t+1}|S_{t+1}) \rho_{t+1} \right) + \gamma (1 - \pi(A_{t+1}|S_{t+1})) V(S_{t+1})$$

This avoids the high variance of pure importance sampling while allowing off-policy learning.

12 Deep Reinforcement Learning

12.1 Deep Q-Networks

Definition 12.1 (DQN (Deep Q-Network)). Use a deep neural network to approximate the Q-function. The network takes state s as input and outputs Q-values for all actions:

$$Q_\theta(s, a) = \text{NeuralNetwork}_\theta(s)[a]$$

This allows generalization across states and scales to complex state spaces. The basic DQN algorithm uses:

- Experience replay: Store and sample transitions from a replay buffer
- Target network: Separate network for computing target values, updated periodically
- ϵ -greedy exploration

12.2 Advanced DQN Methods

Definition 12.2 (Prioritized Experience Replay (PER)). Instead of uniform sampling from the replay buffer, sample transitions proportionally to their TD error:

$$\text{Priority}(i) = |\delta_i| + \epsilon$$

Transitions the agent has difficulty learning (large TD error) are replayed more often. This focuses learning on challenging examples while improving sample efficiency.

Definition 12.3 (Multi-Step Q-Learning in Deep RL). Instead of one-step bootstrapping, use n-step returns:

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n Q_\theta(S_{t+n}, \arg \max_a Q_\theta(S_{t+n}, a))$$

This provides better-balanced targets than pure TD or pure MC, leveraging the first n real transitions while bootstrapping thereafter.

Definition 12.4 (Distributional Reinforcement Learning). Instead of learning a single value estimate, learn the full distribution over returns. The Categorical DQN maintains atoms of the return distribution:

- Support: $Z = \{z_1, z_2, \dots, z_N\}$ equally spaced atoms
- Probability: $p_\theta(s, a) = [p_1(s, a), \dots, p_N(s, a)]$ probabilities for each atom
- Value: $Q_\theta(s, a) = \sum_i p_i(s, a) z_i$

Learning the distribution provides richer information about uncertainty and can improve exploration and robustness.

12.3 Gaussian Policies for Continuous Control

Definition 12.5 (Gaussian Policy). For continuous action spaces, use a Gaussian policy parameterized by mean and variance:

$$A_t \sim \pi_\theta(\cdot | S_t) = \mathcal{N}(\mu_\theta(S_t), \sigma_\theta^2 I)$$

The policy gradient for a Gaussian policy is:

$$\nabla_\theta \log \pi_\theta(a|s) = \frac{1}{\sigma^2} (a - \mu_\theta(s)) \nabla_\theta \mu_\theta(s) - \frac{\sigma'_\theta(s)}{\sigma_\theta(s)}$$

The variance (or log-variance) is often learned as well to explore effectively (high variance early in learning, decreasing as policy improves).

13 Final Remarks

These lecture notes have covered the major theoretical foundations and practical algorithms in reinforcement learning, from dynamic programming to deep neural network approximation. The field continues to evolve rapidly, with new algorithms combining these core ideas in novel ways. Understanding these fundamentals provides a strong foundation for continuing into more advanced topics and research.